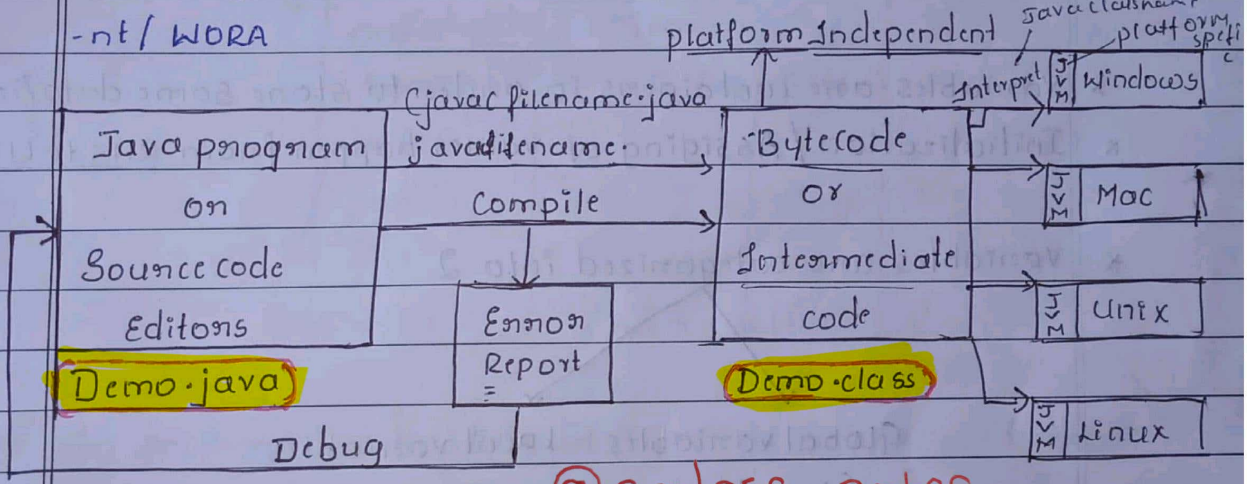
papergrid

Date: 21/01/20

Java

* Java is a high level programming language.
* Programming Language is a medium to interact with s/m
* Highlevel language is a language in a normal English i.e Human understandable form
* James Gosling was a person who introduced Java
* The company which started java is SunMicros/m (system)
* Currently Java is owned by oracle.

Working of a Java Program / How is java platform independe -nt / WORA



@codees_notes

JVM : Java virtual Machine (Platform dependent)

WORA : Write Once Run Anywhere

Bytecode is an intermediate which is neither low-level nor highlevel language so it uses jvm → ① convert to machine level ② Execute line by line

* Firstly we build the java program using editors and save it with the extension .java

* Once we are done developing the program we need to compile it

Scanned with CamScanner

papergrid

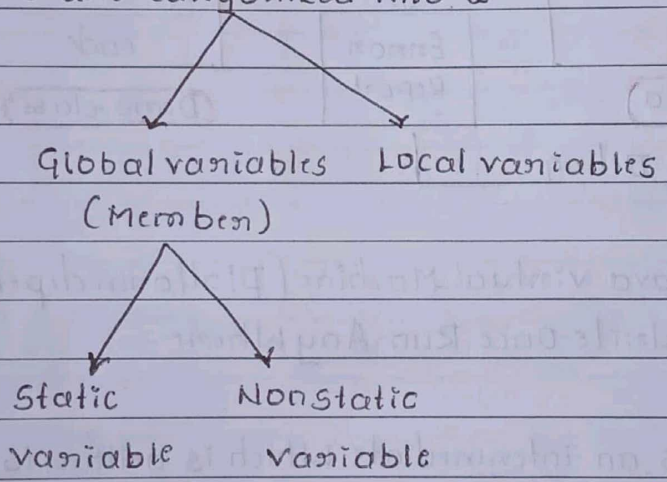
Date: / /

* compilation is a process in order to check if there are any errors in my java program or not
* If compilation is unsuccessful we get error report based on error report we need to debug the program
* If compilation is Successful we generate bytecode which is intermediate code / platform independent code
* Extension of all bytecode is .class
* This bytecode can be executed on all platform i.e all operating sy

Variables

* Variables are containers in order to store some data/inform
* Initialization / Assiging of values happen from RHS to LHS

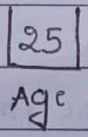
* Variables are categorized into 2



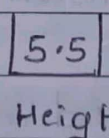
@codees_notes

Example : Age = 25

LHS RHS



Height = 5.5



V => LHS RHS

Datatypes

@codees_notes

* Data types are used to indicate or specify the type of data stored into a variables
* Datatypes are categorized into 2
  1) primitive Datatype
  2) Non primitive Datatype
* In order to store non decimal numeric values we have the following datatypes
* The difference between those datatypes is Memory size

Data types	Memory Size	
	Bytes	Bits
byte	1	8
short	2	16
int	4	32
long	8	64

* In order to store decimal numeric values, we have the following datatypes

Data-T	Memory size	
float	4	32
double	8	64

widely used and default datatype for decimal.

* In order to store true / false we have following datatype

Data-T	Memory size	
	Bytes	Bits
Boolean	1	8

@codees_notes

* In order to store a single character, we make use of char
* char data should be enclosed within 'single quotes(' ')
* The Memory size of char is 2 bytes = 16bits

Note : All the above mentioned 8 data types are together called as primitive data types

String : It is a dataStri type to store a sequence of characters

* String data has to be enclosed within "double quotes"

Note : Java is CaseSensitive where in lowercase letters are not equivalent to upper case letters/values (a ≠ A)

22/01/2020

Variable Declaration

Syntax:

```
datatype VariableName;
int age;
double salary;
```

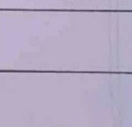
@codees_notes

Variable Initialisation

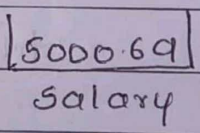
Syntax :

```
VariableName = value;
```

age = 20;



salary = 5000.69



Variable Declaration & Initialization

Syntax :

```
datatype VariableName = Value;
```

Initialization

Declaration

```
boolean x = true;
          false;
```

Date: / /

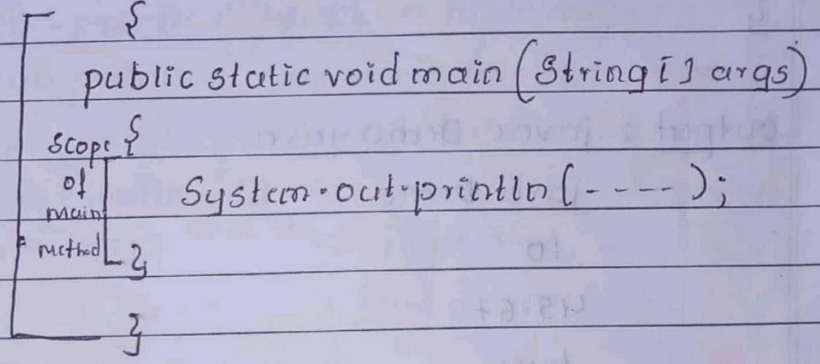
String subject = "java";
= "ijsp2020"; (string can store number, character but it should be within " " quoted)
char gender = 'M';

Structure of a java program:

1) Class
2) Main method
3) Print Statement

@codees_notes

① class : Class className



The filename should be same as classname during file saving className.java

Ex:1) Class Firstprogram
{
public static void main (String[] args)
{
System.out.println ("Hello world");
}
}

@codees_notes

Oulput: Open cmd
cd desktop // cd:- change directory
cd java programs to enter to
javac Firstprogram.java -(compiln)
java Firstprogram -(interprtn) the save files
Hello world!!!

Ex:2)
```
Class Demo
{
Public static void main (String[] args)
{
System.out.println (10);
System.out.println (45.67);
System.out.println (true);
System.out.println ('2');
System.out.println ("Ijsp@2020");
}
}
```

Output : javac Demo.java

```
java Demo
10
45.67
true
2
Ijsp@2020
```

@codees_notes

Ex3) Write a java progrnam to follow the below statement / Scenarios

i) Create class called as student

ii) Define main method

iii) Under main method initialize 2 variables called as name and age entering those respective values

→
```
Class student
{
public static void main (String [] args)
{
   String name: "Bhagya";
    int age = 23;
   System.out.println ["name"];
    system.out.println (age);
}
}
```

O/P

Bhagya

23

papergrid

Date: / /

Note: In java, in order to perform Concatenation we make use of '+' operator

Q)
```
class Employee
{
public static void main (String[] args)
{
 int id = 101;
 String name = "Jerry";
 double salary = 123.45;
System.out.println("Employee Id:" +id);
System.out.println("Employee name is:" +name);
System.out.println("Employee Salary =" +salary);
System.out.println(id +" " +name +" " +salary);
}
```

@codees_notes

Output

```
javac Employee.Java
java Employee
Employee Id=101
Employee Name is : Jerry
Employee Salary = 123.45
101 Jerry 123.45
```

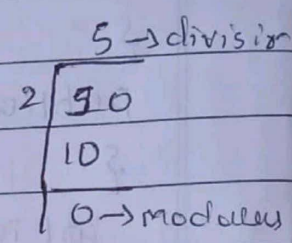
23/1/2020

Operators

1) Arithmetic Operators
2) Assignment Operators
3) Relational / Conditional / comparsion Operators
4) Logical Operators
5) Unary Operators

1) **Arithmetic Operators**

+ : Addition
− : Substraction
* : Multiplication
/ : Division
% : Modulus



Ex: 10/2 = 5 10%2 = 0

Example:

```
Class ArithmeticOperators
{
public static void main (String[] args)
 {
    int x=10;
    int y=20;
    int sum = x+y;
    int diff = x-y;
 System.out.println("Sum = "+sum);
 System.out.println(" Difference is " + diff);
 System.out.println (y*5);
 System.out.println (30/3);
 System.out.println (30%3);
```

Output:-

```
Sum = 30
Difference is = -60
100
10
0
```

@codees_notes

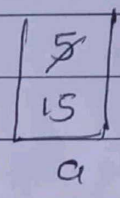
Date: / /

2) Assignment operators

=
+=
-=
*=
/=
%=

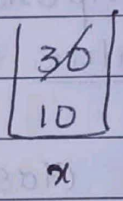
@codees_notes

```
int a=5;
a= a+10  or  a+=10
a=15
```



@codees_notes

```
int x=30
x-=20
x=x-20
x=30-20
x=10
```



Ex:

```
Class Assignment Operator
{
public static void main (String[] args)
{
 int x=10;
 System.out.println("value of x is :" +x);
    x+=20;
 System.out.println("value of x is :" +x);
 System.out.println("= = = = =");
 int a= 6
 system.out.println("value of a is:" +a);
    a*=5;
 system.out.println("value of a is:" +a);
}
}
```

O/P

Pathname → cmd
= Assigning
== comparsion

papergrid

Date: / /

Output

Value of x is 10
value of x is 30
= = = = = =
value of a is 6
value of a is 30

3) Relational / conditional / comparsion Operators

< : lesser than
> : Greater than
<= : lesser than or equal to
>= : Greater than or equal to
== : Equals to
!= : not equal to

Note : Comparsion Operators will always return boolean values i.e (true/false)

Ex:

```
Class comparsion operator
{
public static void main (String[] args)
{
    int x=10;
    int y=20;
```

@codees_notes

```
    boolean result1 = x<y;
    boolean result2 = y>x;
System.out.println(result1 + "\t" + result2);
System.out.println("= = = = ");
System.out.println (x<=10);
System.out.println (3>=4);
System.out.println ("- - - - ");
System.out.println (x==10);
System.out.println (y==30);
System.out.println ("- - - - ");
System.out.println (x!=10);
System.out.println (y!=30);
```

O/P

true true
---
true
false
---
true
false
---
false
true

4) **Logical operators**

AND => &&
OR => ||
NOT => !

return => boolean { true = 1, false = 0 }

AND		&&
T	T	T
T	F	F
F	T	F
F	F	F

OR		||
T	T	T
T	F	T
F	T	T
F	F	F

NOT	!
T	F
F	T

Ex: Class logical operators

```
{
public static void main (String[] args)
{
int x=10;
int y=20;
boolean result1 = x<y && y>x;
boolean result2 = x<y && x==1;
System.out.println (result1);
System.out.println (result2);
System.out.println ("---");
System.out.println (1<2||2>10);
System.out.println (2<1||2==3);
System.out.println ("---");
System.out.println (!true);
System.out.println (!false);
System.out.println ("---");
System.out.println (!(1<2));
}
}
```

@codees_notes

@codees_notes

O/P

```
true
false
---
true
false
---
false
true
---
false
```

5) **Unary Operators**

++ (increment by 1)

-- (Decrement by 1)

++ → pre increment
   → post increment

-- → pre decrement
   → post decrement

Ex:
```
Class Unary
{
public static void main (String[] args)
  {
    int x = 5;
  System.out.println ("x:" +x);
    x++;
  System.out.println ("x:"+x);
    ++x;
  System.out.println ("x:"+x);
    x--;
  System.out.println ("x:"+x);
    --x;
  System.out.println ("x:"+x);
  }
}
```

@codees_notes

Output ⇒
```
x:5
x:6
x:7
x:6
x:5
```

① `int x= 10;`
`int y= x++;`

post increment → First assign, then increment

x [10 → 11] y [10]

② `int a= 5`
`int b= ++a;`

pre-increment → First increment, then assign

a [5 → 6] b [6]

Date: / /

(3) int i=2
j=i--;
post-decrement
First assign, then decrement
i [1] j [2]

(4) int a=5
int b=++a;
pre-increment
First increment, then assign

@codees_notes

(4) int p=50;
int q=--p;
pre-decrement
First decrement, then assign
p [50 49] q [49]

Ex) class Unary
{
psvm (String[] args)
{
int q=123;
int w=q++;
System.out.println(q+" "+w);

System.out.println("-----");

int o=444;
int p=--o;
System.out.println(o+" "+p);

}
}

@codees_notes

Decision / conditional statements

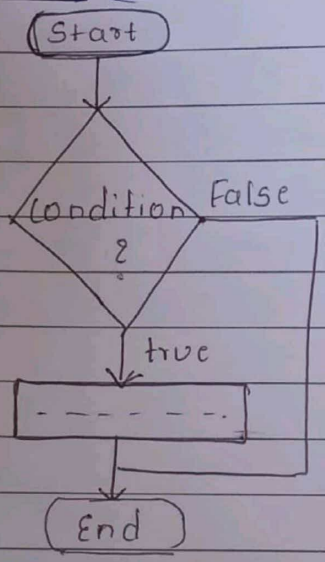
These statements are used to take some decision based on the condition specify. The different decision statements are as follows

1) Simple if
2) if else
3) if else if
4) Nested if
5) Switch

(1) Simple if

Simple if is a decision making statements where in we execute a set of instructions only if the condition is true

Ex: Flow chart



Syntax:

```
if (condition)
{
  statement 1;
  - - - - -
  statement n;
}
```

Set of instructions

Ex:-

```
class SimpleifDemo
{
psvm (String[] args)
{
 System.out.println ("START");
}
    int a=10;
    int b=10;
```

@codees_notes

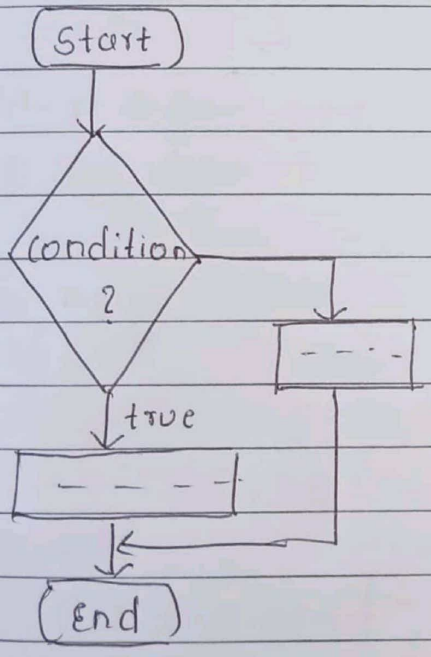
```
if (a<=b)
{
system.out.println (a+ " is less than or equal to" +b);
}
system.out.println ("End");
}
}
```

(2) **if else :-**

@codees_notes

x if else is a decision making statement where in, if the condition is true we execute if block. Otherwise if the condition is false we execute else block:-

flow chart



Syntax :

```
if (condition)
{
statement 1;
- - - -
statement n;
}
else
{
-- --
}
```

set of instructions

Ex

```
class IfelseDemo
{
  psvm(String[] args)
{
   s.o.pln("START");
   int x=100

   if (x<=10)
   {
s.o.pln(x+" is lesserthan or equal to 10");
   }

  else
   {
   s.o.pln(x+" is gorcater than 10");
  }
   s.o.pln(" ------ ");

 if(true)
 {
  s.o.pln("HI");
 }
else
 {
  s.o.pln("BYE");
}
 s.o.pln("----")

  if(false)
 {
  s.o.pln("WELCOME");
 }
```

@codees_notes

```
else
{
S.O.pln("Thank you");
}
S.O.pln("END");
}
}
```

Op: START
100 is greater than 10
- - - - - - -
Hi
- - - -
Thank you
END

Q) Write a java program to find a number is +ve or -ve

→
```
class Number
{
psvm(String[] args)
{
    int x=5;
    if(x>0)
  {
    S.O.pln("x is a positive number");
  }
    else
    {
      S.O.pln("x is a negative number");
    }
}
```

@codees_notes

2) Write a java program to find a number is even or odd

→
```
class Number1
{
 psvm (String[] args)
  {
    int num=4;
    if (num % 2==0);
    {
     s.o.pln (num+" is a even number");
    }
     else
     {
      s.o.pln (num+" is a odd number");
     }
}
```

@codees_notes

3) Write a java program to find maximum of 2 numbers

→
```
class Number2
{
 psvm (String[] args)
  {
    int x=5;
    int y=10;

    if (x>y)
{
 s.o.pln (x+" is a langer than"+y);
}

else
 {
  s.o.pln(x+" is a lesser than"+y);
 }
}
```

@codees_notes

27/1/2020

3) **if-else-if**

if-else-if is a decision making statement where in we can check multiple conditions

Syntax:

```
if (condition)
{
  ---
  --
}
else if (condition)
{
  ----
}
else if (condition)
{
}
else
{
}
```

(optional)

Ex:

```
class IfElseIfDemo
{
  psvm(String[] args)
  {
    if num=250;
    if (num<=10)
    {
      S.O.pln(num+" is lesser than or equal to 10");
    }
```

@codees_notes

```
else if (num<=20)
 {
 S.O.pln (num+" is lesser than or equal to 20");
 }
else if (num<=30)
 {
 S.O.pln (num+" is lesser than or equal to 30");
 }
 else
 {
 S.O.pln ("Above conditions did not match");
  }
 }
}
```

@codees_notes

O/P : Above condition did not match

0-34-F
35-59-F
60-100-F

Ex

```
class IfElseIfDemo1
{
 psvm (String[] args)
 {
  int marks=25;
  if(marks>=0 && marks<=34)
  {
    S.O.pln("fail");
  }
  else if (marks>=35 && marks<=59)
  {
    S.O.pln (" First class");
  }
```

```
else if (marks >=60 && marks <=100)
{
    s.o.pln("FCD");
}
else
{
    s.o.pln("Invalid Marks");
}
}
}
```

@codees_notes

O/P : Fail

(4) **Nested if :**

Nested if is a decision making statement where in, one if is presented inside another if condition

Syntax:

```
if (condition)
{
  if (condition)
  {
    ---
    ---
  }
  else
  {
  }
```

Ex (1)

```
class NestedIfDemo
{
psvm (String[] args)
{
  char id = 'b';
  int password = 123;

  if (id == 'a');
  {
    S.O.pln (" user id is valid");
      if (password == 123)
      {
        S.O.pln (" password is valid ");
        S.O.pln (" login is Successful");
      }
```

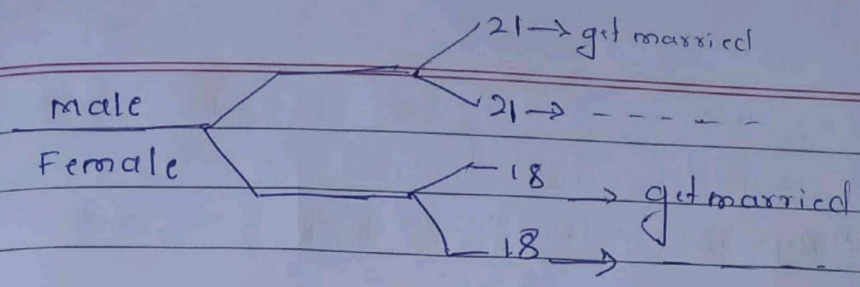
@codees_notes

```
  else
    {
      S.O.pln (" password is invalid");
              (" login is Unsuccessful");
    }
}
else
    {
      S.O.pln (" user id is Invalid");
              (" login is Unsuccessful");
    }
}
```

Output :- (1) Userid is Invalid
login is Unsuccessful

(2) User id is valid
Password is valid
login is Successful

Ex




```
class NestedIfDemo1
{
  psvm(String[] args)
  {
    char gender='M';
    int age = 24;

    if(gender=='M');
    {
      s.o.pln("Male");
        if(age>=21)
        {
          s.o.pln("Age is :"+age);
          s.o.pln("Get married & hopefully stay happy");
        }
        else
        {
          s.o.pln("age is :"+age);
          s.o.pln("Have patience");
        }
    else if(gender=='F')
    {
      s.o.pln("Female");
        if(age>=18)
        {
          s.o.pln("Age is :"+age);
        }
```

@codees_notes

```
else
{
s.o.pln ("Age is :" +age);
        (                )
}
}
else
{
s.o.pln ("Gender is Invalid");
}
}
```

(2) Gender is Invalid

o/p :- Gender is Male
age : 21
Get married & hopefully stay happy
stay happy

8) Write a java program to find largest of 3 numbers.

→

```
class largestofthreeNumbers
{
  psvm(string[] args)
  {
    int a =10;
    int b= 5;
    int c= 3;
  s.o.pln("a:"+a+" b: "+b+" c :"+c");
    if (a>b)
    {
      if (a>c)
      {
        s.o.pln("a is largest");
      }
```

@codees_notes

```
else
{
 s.o.pln("c is largest");
}
}
else if(b>c)
{
 s.o.pln("b is largest");
}
else if(c>b)
{
 s.o.pln("c is largest");
}
 else
 {
  s.o.pln("Invalid");
 }
}
}
 s.o.pln("----");

 if(a>b && a>c)
 {
  s.o.pln("a is largest");
 }
else if(b>a && b>c)
 {
  s.o.pln("b is largest");
 }
else if(c>a && c>b)
 {
  s.o.pln("c is largest");
 }
```

a b c
a c
a or c

@codees_notes

```
else
 {
 s.o.pln("Invalid");
  }
 }
}
```

28/1/2020

(5) **Switch statement**:-

Switch is a conditional statement generally used for character comparsion

Syntax:-

```
switch (choice/input)
{
 case 1: -----
        break;
 case 2: -----
        break;
  ---
   :
 case n: ----
 default:
```

@codees_notes

Ex:-

```
class switchDemo
{
psvm (String[] args)
 {
   int choice=3;
     switch (choice)
 {
case 1: s.o.pln("In case 1");
        break;
case 2: s.o.pln("In case 2");
        break;
case 3: s.o.pln("In case 3);
        breaks;
  default: s.o.pln("Invalid choice");
 }
}
}
```

O/p:- In case 3

Break: is a keyword which is used to transfer the control outside the currently executing block

Ex:

```
class monthvalidation
{
psvM (String [] args)
 {
   S.O.pln ("start");
   char month = 'Z';
    switch (month)
    {
     case 'J' : S.O.pln ("In January");
                break;
     case 'F' : S.O.pln ("In February");
                break;
     case 'M' : S.O.pln ("In March");
                break;
     default : S.O.pln ("Invalid month");
      }
      S.O.pln ("End");
    }
}
```

@codees_notes

Output: start
Invalid Month
End